VISIONVOICE AI

Python-Based Multimodal Chatbot — Project Report (Alternate Edition)

A separate alternate-edition project report

Project snapshot

Item	Details
Project	VisionVoice AI
Frontend	Streamlit (Python)
Backend	FastAPI + Uvicorn
AI provider	Google Gemini / google-genai
New feature	Balanced, Brief, Detailed response styles
Upload limit	10 MB by default

1. Abstract

VisionVoice AI is a Python-only multimodal chatbot that accepts typed questions, images, and audio clips. It combines a Streamlit interface, a FastAPI backend, and Google's Gemini model through the official Google GenAI Python SDK. This alternate edition introduces selectable response styles—Balanced, Brief, and Detailed—while retaining the original project's lightweight, stateless architecture.

2. Introduction

People often need to ask questions about more than plain text. A useful assistant should also interpret a picture or help understand an audio recording. VisionVoice AI brings these input types into one simple interface and returns a text response.

3. Problem Statement

Separate tools are often used for text questions, image interpretation, and audio summarization. This project demonstrates a unified Python application that routes each input type through a common backend while validating uploaded media and keeping provider credentials out of the user interface.

4. Objectives

The objectives are to provide text chat, image question answering, audio analysis, selectable response detail, clear loading and error feedback, retry-friendly input handling, and server-side validation of uploaded files.

5. Scope

The application supports text prompts, JPG/JPEG/PNG/WebP images, and common audio formats accepted by the configured Gemini model. It is designed as a local educational prototype. User chat history is held in Streamlit session state; uploaded media is processed in memory and is not stored in a project database.

6. Technology Stack

Python 3.10+, Streamlit, FastAPI, Uvicorn, Google GenAI Python SDK, Pillow, Requests, python-dotenv, pytest, and HTTPX. The frontend and backend are both Python; there is no custom JavaScript, HTML, CSS, or Node.js application.

7. System Architecture

The Streamlit frontend collects the selected modality, prompt, optional file, and answer style. It sends a multipart HTTP request to FastAPI. The backend checks the request, upload size, MIME type, extension, and media signature where applicable. The provider module then calls Gemini and returns generated text to the frontend.

8. Main Modules

frontend/app.py implements the chat interface, media preview, answer-style selector, loading indicators, and error display. backend/main.py exposes health and chat endpoints and validates uploads. backend/provider.py encapsulates the Gemini SDK call. backend/config.py reads environment configuration. tests/test_api.py verifies API behavior using mocked provider responses.

9. Answer Style Feature

Balanced asks for a clear, helpful response; Brief asks for the most important points only; Detailed asks for a fuller explanation with steps and examples when useful. The selected style is included as a natural-language instruction in the prompt, so the backend API contract remains simple.

10. Setup and Execution

Create and activate a virtual environment, install requirements.txt, copy .env.example to .env, and set GEMINI_API_KEY. Start the API with `uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000`. In a second terminal, run `streamlit run frontend/app.py`. Open the local Streamlit address printed in the terminal.

11. Configuration

GEMINI_API_KEY stores the provider secret; GEMINI_MODEL selects the model and defaults to gemini-2.5-flash; MAX_UPLOAD_MB sets the upload limit and defaults to 10; BACKEND_URL sets the FastAPI address and defaults to http://127.0.0.1:8000.

12. Security and Privacy

The Gemini key is read by the backend from environment configuration and is not sent to the Streamlit client. Uploads are checked before provider use. Media is not written to a local uploads directory or database by this prototype. Requests are sent to Google Gemini for inference, so users should avoid private or sensitive files and review the provider's current data-handling terms.

13. Testing and Verification

Automated API tests were run with the Gemini provider mocked. Result: [32m.[0m[32m.[0m[32m.[0m[32m.[0m[32m.[0m[32m.[0m[32m.[0m[32m.[0m[32m [100%][0m [32m[32m[1m7 passed[0m[32m in 0.46s[0m[0m Spreadsheet runtime warmup failed during python startup Traceback (most recent call last): File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 26, in warm_spreadsheet_runtime_on_startup File

"/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 785, in warm_spreadsheet_runtime File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 718, in _warm_feature_flows File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 509, in _warm_drawing_and_image_flows File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/interface/models.py", line 32861, in bring_to_front File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote.py", line 764, in __call__ File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client.py", line 175, in call artifact_tool.rpc.client.RemoteError: Element ordering requires an owning element collection.. These tests validate local API behavior and do not prove that a live Gemini key, model access, or every audio codec works. Before demonstration, manually test text, image, audio, oversized and invalid uploads, backend-offline behavior, and provider errors with a valid API key.

14. Limitations

The prototype has no user accounts, database, persistent chat history, production authentication, rate limiting, monitoring, or deployment-grade privacy controls. Supported audio formats may vary by provider/model. Network access and a valid Gemini API key are required for live responses.

15. Future Enhancements

Possible improvements include conversation export, optional user authentication, configurable system prompts, model selection, speech-to-text display, richer file metadata, deployment hardening, rate limiting, and a persistent store with explicit user consent.

16. Conclusion

VisionVoice AI demonstrates a compact Python-only approach to multimodal interaction. Its alternate-edition response-style selector adds a small user-facing customization while keeping the core architecture understandable and suitable for an educational lab.

Appendix — Run commands

• python -m venv .venv

• pip install -r requirements.txt

• Copy .env.example to .env and add GEMINI_API_KEY

• uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000

• streamlit run frontend/app.py

• python -m pytest -q